PolicyHub

Michelle Nguyen (Frontend Designer)
Fernando Rivera-Armstrong (Database Designer)
Tomas Rohatynski (Team Lead)
Joseph Cowles (Project Manager)

Abstract

PolicyHub aims to establish a centralized database for health insurance policy coverage with version history, to help hospital billing staff navigate the intentionally obfuscated documents published by insurance companies.

Problem Statement

Currently, hospitals need to maintain large billing departments in order to keep up with the ever-changing insurance policy coverages provided by insurance companies. Insurance companies intentionally obfuscate their policy information in order to hinder hospitals' ability to bill them for procedures that the hospitals perform. The policy can change at any time, and there is no announcement made when they do, and usually there is no version history either. Every time a hospital wants to bill an insurance company, they must send a form to that company to request their coverage information for a certain CPT (Current Procedural Terminology) code on a certain date that the operation was/is performed.

Solution

Our solution to overcoming this is a centralized database that is populated by a 3rd party, that hospitals can access in a uniform way (using an API or website) to retrieve the information they need on-demand. This will allow hospitals to slim down their billing departments, improve billing turnaround time, and save costs for their patients.

Benefits

The most important advantage of a central database versus an Excel file, is that changes can reflect immediately instead of having to send a new copy of the spreadsheet. This is important in the use-case of this project, because the insurance policy coverages can change at any time without any notice, so the hospitals need to be informed of the change as quickly as possible. Another advantage is that by accessing the database through an API, it allows for the data to be centralized, and thus minimizes the effort needed to push the data and updates to users since they can always access the latest version when they need to. Furthermore, with the ability to set up read-replicas of the data (the data will be updated periodically, but accessed very frequently), it is possible for a huge number of customers to access it.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Comparison with Excel

The database can be queried so that only policy information for a specific date is returned, whereas setting up an Excel spreadsheet in such a way would require a lot of work to make it foolproof in order for the users to get the correct information. This is a common use case for hospitals since they need to be able to quickly access policy information for a given Date of Service (DOS). This project simplifies data access for hospital billing staff, even when the data has been intentionally obfuscated.

Requirements and Target Use

The database would be maintained by a private company that would license use of the database to users. The private company would ensure that the information stored in the database is accurate. A web scraper would be the primary data source in a production application, however, web-scrapers are inherently fragile, so admin oversight is needed. Through the admin portal, the database maintainers can add, supersede, and delete policies to make sure that the information is accurate.

Hospital Billers

- Hospital billing staff use the system to look up accurate insurance policy coverage information for a specific Date of Service (DoS) in order to bill the patient accurately.
- They need fast and flexible search capabilities using parameters such as CPT code, CPT category, or procedure name.
- These users would have read-only access, allowing them to view the data without modifying or deleting records.

Database Administrator (DBA)

- Database administrator is responsible for managing and maintaining policy and coverage data within the system.
- They would have permission to add new policies, update or supersede existing policies as coverage rules change, and remove outdated or incorrect records.

Software Developer

- Software developers use the system to maintain and hence the platform's functionality and performance.
- They would have technical access to support development and maintenance tasks (such as modifying database schemas) but do not manage policy data as part of normal operations.

List of Relations

- Relation: Insurance_Provider
  - (PK) Insurance_Provider_ID
  - Name
- Relation: Procedure
  - (PK) ID
  - (FK) User_ID
  - (FK) Procedure_Type_ID
  - Date_Of_Service
- Relation: Procedure_Type
  - (PK) Procedure_ID (CPT Code)
  - Name
  - Description
- Relation: Policy
  - (PK) ID
  - (FK) Insurance_Provider_ID
  - Name
  - Deductible
- Relation: Policy_Coverage
  - (PK) ID
  - (FK) Policy_ID
  - (FK) Procedure_Type_ID (CPT Code)
  - Description (long md file)
  - Link_to_Original
  - Start_Date
  - End_Date
- Relation: User
  - (PK) Username
  - Email
  - Password (hashed)
  - (FK) Role_ID
- Relation: Role
  - (PK) Role_ID
  - Role_Name
  - (FK) Permission_ID
- Relation: Permission
  - (PK) Permission_ID
  - Permission_Access
  - Permission_Name

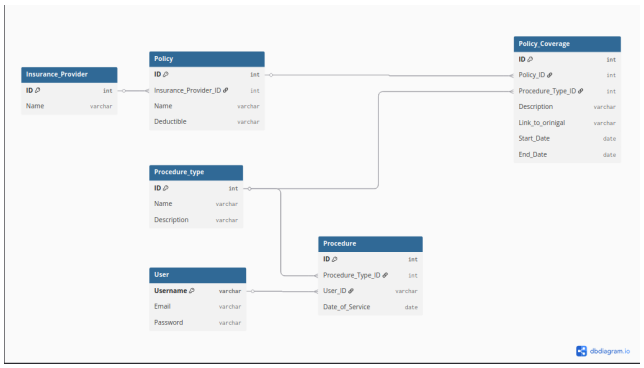

Figure 1: Relations Part 1

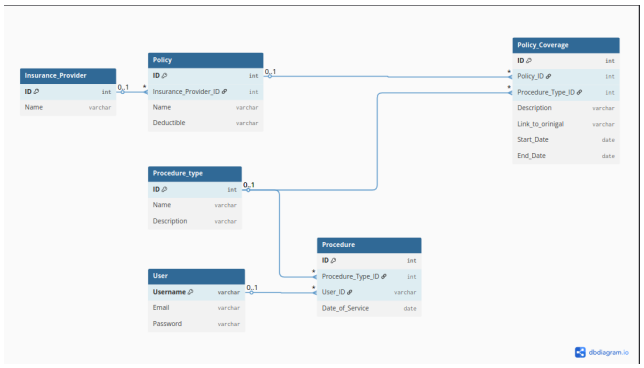

Figure 2: Relations Part 2

Data

For real world applications, the data would be created and maintained by a web scraper, however, for our project, we wanted to avoid the added complexities involved with building one. For that reason, we are opting to use an LLM to generate policy data. We will give an LLM some CPT codes along with a description and tell it to generate an insurance policy that covers them. One concern that we had was how accurate the emulated policy information would be without grounding, which is why we will provide the LLM with a real insurance policy in the system prompt to ensure that the data is realistic. For user data, we will use a simple python program to generate the data. Operation data, including CPT codes, is commonly available and we will use a python script to format a CSV that we found with the data.

Interface

Primary System Screen (Core User Interface)

Login Screen Allows authorized users to securely sign in to the system using their credentials; ensuring only permitted users can access system features and data.

- Users input their organization-issued email/username and password, and the system authenticates them and grants role-based access to authorized features.
- As a B2B service, accounts would be provisioned through hospital onboarding rather than open self-registration.

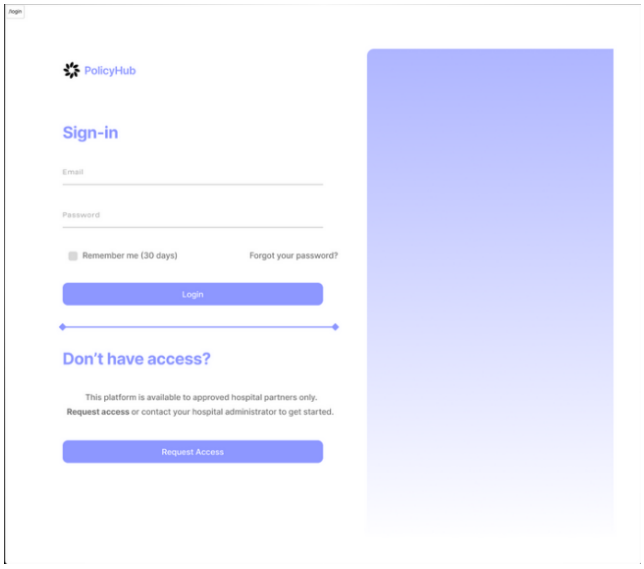

Figure 3: Login Screen

Search Screen Provides fast and flexible search tools where users can look up specific policy information by CPT code, procedure name, category, or other advanced filters.

- Users input search criteria such as CPT code (text), procedure name (text), category (dropdown selection), insurance provider (dropdown selection), or Data of Service (date). The system then outputs a list of matching insurance policies.

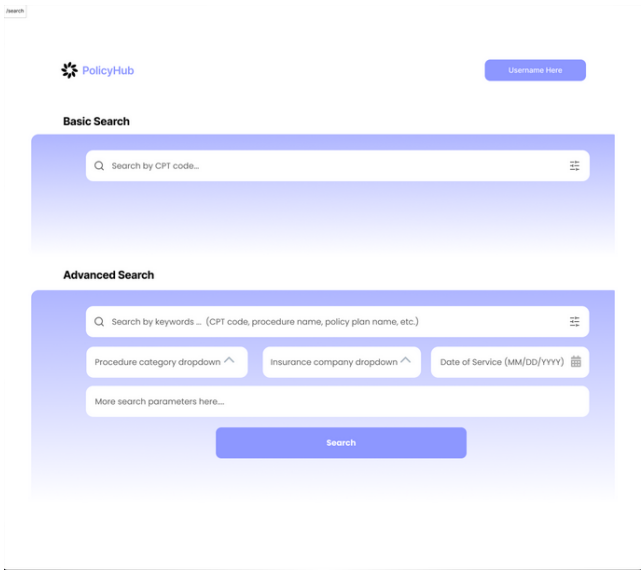

Figure 4: Search Screen

Policy Details Screen Displays detailed coverage information for a selected policy, including procedures, coverage status, and other information in a clear, read-only format.

- Users select a policy from the search results. The system outputs detailed, read-only coverage information and relevant policy details.

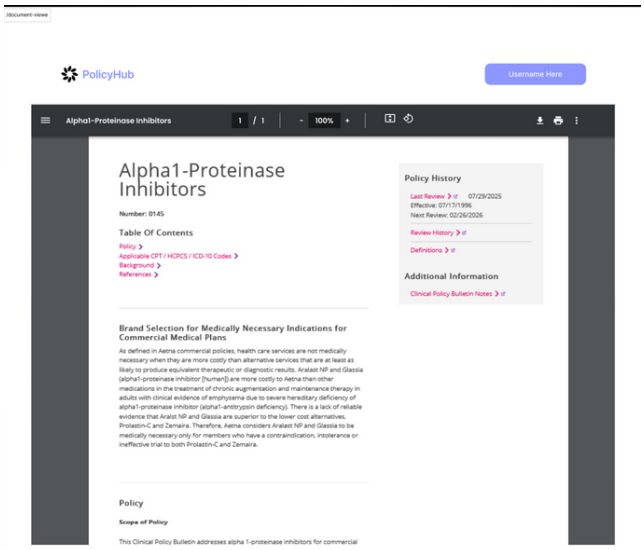

Figure 5: Policy Details Screen

Extended Screen (Future Enhancements)

- Database Management: allows DBA to manage data such as adding, updating, deleting or superseding insurance policy records and coverage rules.
- User Management: allows DBA to manage user accounts, assign roles and control permission to maintain security and proper access privilege.